

Computer Vision Final Project

Counting of the people in the Plaza de Ferrandiz i Carbonell and access to the University

Giorgi Gogsadze
27.06.2025

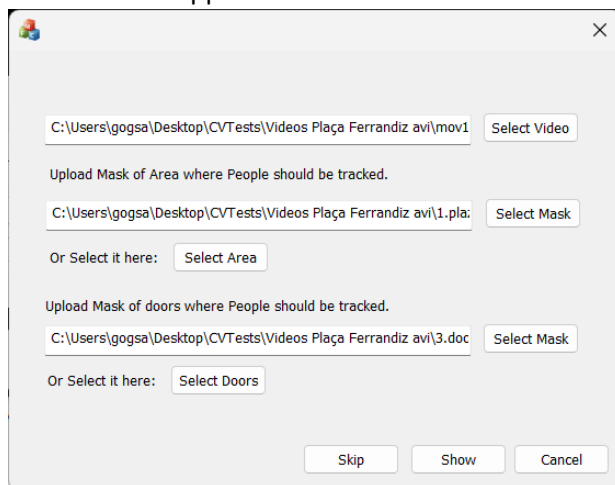
The goal of the project

The project is about analyzing a video captured by a webcam located in the Ferrándiz building, overlooking the square near the entrance to Carbonell through the Library. There was two main tasks:

1. **People Counting in the Square:** Detect and count the number of people present in the square at any given time.
2. **Entry and Exit Monitoring:** Track the number of people entering and exiting the Carbonell building through the visible doors in the video, in order to estimate the current occupancy inside the building.

Used Technology and the Application

- The project is developed as a C++ MFC dialog-based application. OpenCV library is used for main operations.
- The user interface provides several functionalities, including:
 - the ability to upload a video and apply predefined static masks for the plaza and door areas.
 - Alternatively, users can interactively create custom masks within the application.



Static Masks

The masks are necessary so the system can know where to count people or detect entrance/exit. In this project, static masks were used to define and isolate key regions of interest within the video frame: **the doors** and **the square area**.

The decision to use static masks is motivated by these reasons:

- **Lack of Visual Consistency:** The video does not provide any semantic or geometric cues that consistently identify which parts of the frame correspond to doors or to the square, these areas don't have consistent visual features, so they can't be detected dynamically or the detection will waste a lot of computational power and with high probability give errored result.
- **Fixed Camera Setup:** The assignment focuses on a specific static webcam setup, ensuring that the relative positions of the doors and square remain unchanged throughout the entire video. This stability makes it feasible and effective to **manually define static masks** once and reuse them across all frames, without the need for real-time region detection. This makes the use of static masks more practical, reliable, and efficient



Interactive Mask Creation

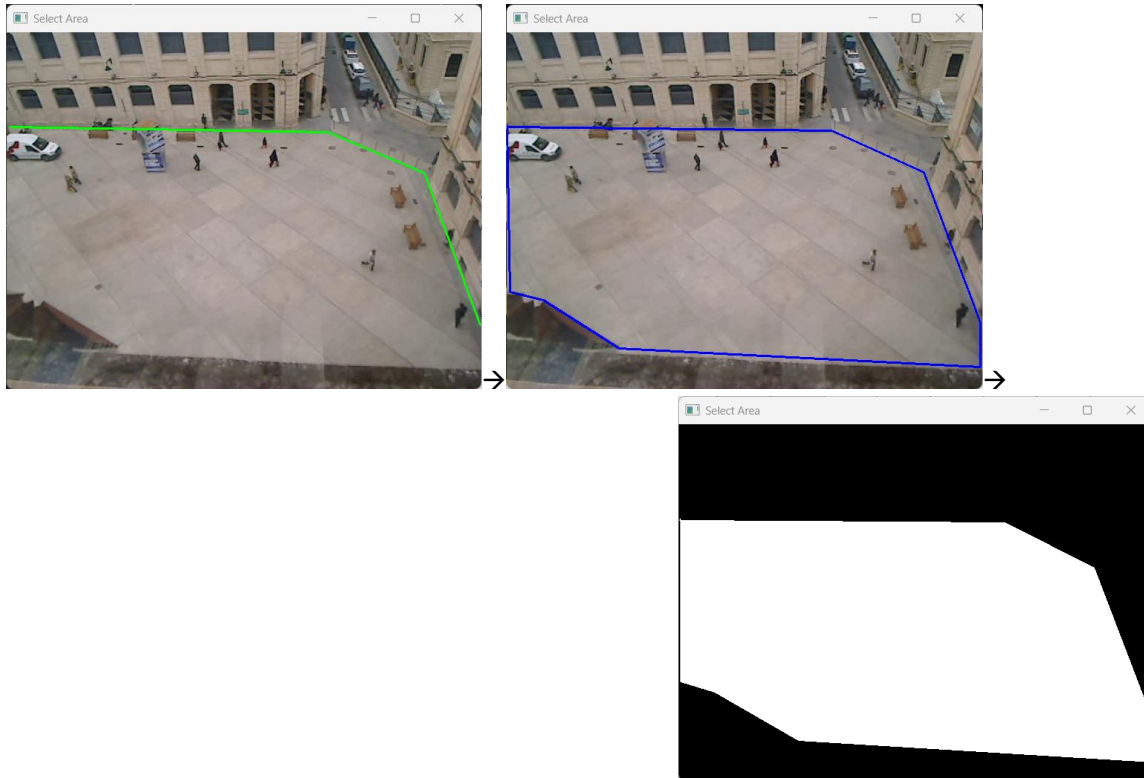
the program allows the users to create the mask themselves using 1st frame of the Video. This is implemented by applying `setMouseCallback("Select Area", selectArea, 0)` function to the frame, that will display the image to the User.

Then it lets user put points and lines on the screen with clicking left button of the mouse and moving it:

```
vector<Point> polygonPoints;
if (event == EVENT_LBUTTONDOWN) {
    polygonPoints.push_back(Point(x, y));
}
if (event == EVENT_MOUSEMOVE && drawing && !polygonPoints.empty()) {
    ...
    line(tempImg, polygonPoints.back(), Point(x, y), Scalar(0, 255, 0), 2);
    ...
}
```

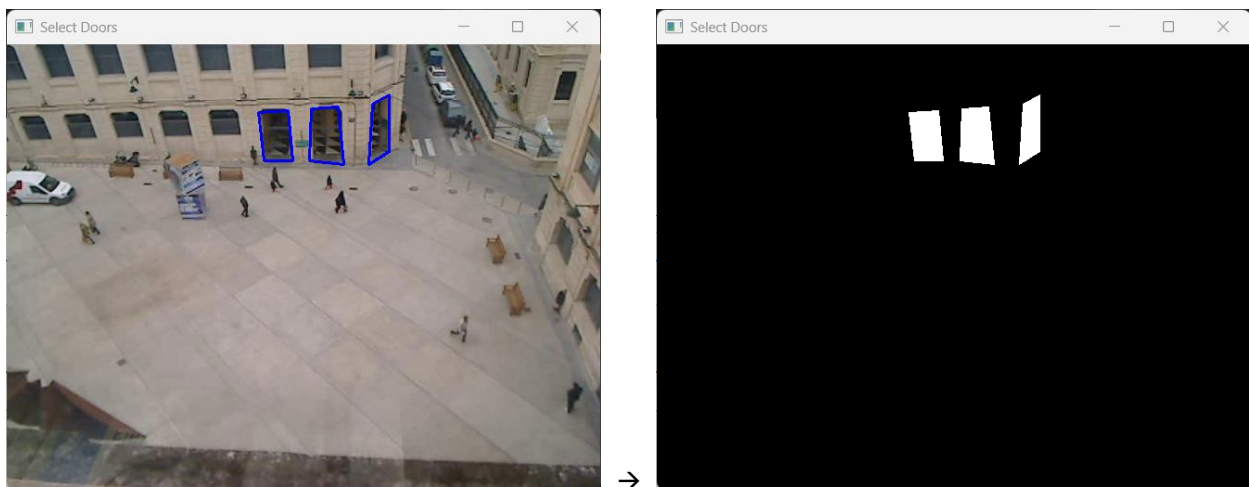
and after this points are connected with lined the program fills the poligon with white color, turning it into binary mask:

```
fillPoly(mask, pts, Scalar(255));
```



The case of doors is similar, with the change that there will be several polygons instead of 1, using vector of vectors:

```
vector<vector<Point>> allPolygons;
```



Of course, it's necessary to clean every mask provided. Example for the area Mask:

```
cvtColor(areaMask, areaMask, COLOR_BGR2GRAY);
threshold(areaMask, areaMask, 1, 255, THRESH_BINARY);
Mat kernel = getStructuringElement(MORPH_RECT, Size(3, 3));
cv::morphologyEx(areaMask, areaMask, cv::MORPH_OPEN, kernel);
cv::morphologyEx(areaMask, areaMask, cv::MORPH_CLOSE, kernel);
```

The Main Logic of the Application

Tracking people

To count people in the square and monitor entry/exit through the doors, the system implements a custom **multi-object tracking** approach based on **background subtraction**, **contour detection**, and **manual association of object tracks** across frames. This section outlines the method used to identify and maintain trajectories of individuals detected in a static webcam video.

1. Background Subtraction

The first step involves detecting **moving objects** in each frame by subtracting the static background. For this purpose, the MOG2 (Mixture of Gaussians) algorithm is used, which allows modeling the background over time and handles illumination changes and slight camera noise.

```
Ptr<BackgroundSubtractor> bgSubtractor;  
bgSubtractor = createBackgroundSubtractorMOG2(500, 16, true);
```

To ensure that the background model is correctly initialized, the system skips the first few frames:

```
Mat fgMask;  
bgSubtractor->apply(mat, fgMask);  
fgMask.setTo(Scalar(255), fgMask == 127);  
// Wait for background to stabilize  
if (i < 10) {  
    i++;  
    return;  
}
```

2. Foreground Mask Processing

To reduce noise and improve contour extraction, the foreground mask undergoes a sequence of preprocessing steps:

1. **Gaussian blur** to suppress high-frequency noise:
2. **Binary thresholding** to ensure a clean binary image.
3. **Morphological operations** to remove small artifacts and fill holes within detected blobs.

```
GaussianBlur(fgMask, fgMask, Size(5, 5), 0);  
threshold(fgMask, fgMask, 25, 255, THRESH_BINARY);
```

```
Mat kernel = getStructuringElement(MORPH_RECT, Size(5, 5));  
morphologyEx(fgMask, fgMask, MORPH_OPEN, kernel);  
morphologyEx(fgMask, fgMask, MORPH_CLOSE, kernel);
```

The program generates a binary image from each frame, clearly highlighting the moving objects:



next step is to extract and analyze these regions.

3. Contour Detection

Contours representing moving objects are extracted from the cleaned mask:

```
vector<vector<Point>> contours;
findContours(fgMask, contours, RETR_EXTERNAL, CHAIN_APPROX_SIMPLE);
```

Contours are filtered based on their area and compactness to discard non-human noise (e.g., small or overly dense shapes):

```
int sizemin = 120;
int sizemax = 9000;
if (contourArea(contour) < sizemin || contourArea(contour) > sizemax)
    continue; // too small or big for a human
Rect bbox = boundingRect(contour);
if (contourArea(contour) / (bbox.width * bbox.height) > 0.7) continue;
// too compact for a human
```

4. Track Representation

Each detected person is tracked over time using a custom structure that stores their bounding box, centroid, motion history, and other properties:

```
struct Track {
    Rect bbox; Point centroid, prevCentroid, middleDown, prevMiddleDown; bool
counted; int id; int missingFrames = 0; Scalar color; int estimatedPeople;
};
```

5. Estimating Number of People in a Track

Because individuals closer to the camera appear larger, the estimated number of people per detection is adjusted based on the area of contour detected and its vertical position (y coordinate). This tells us how close or far the people are from the camera, therefore closer ones have larger area.

```
int estimateN(double area, Point center) {
    int frameHeight = mat.rows;
    int estimateAreaOfPerson = 0;
    if (center.y < frameHeight / 2) {
        estimateAreaOfPerson = 320.0f;
    }
    else {
        estimateAreaOfPerson = 520.0f;
    }
    int estimatedCount = std::round(area / estimateAreaOfPerson);
    return std::max(1, estimatedCount);
}
```

6. Track Association and Update

To maintain consistent tracking, detected contours are matched to existing tracks based on the minimum distance between centroids. It starts with using a threshold of 50 pixels to limit the maximum allowed distance for a match. However, due to variations in video frame rate, object speed, and scene scale, this fixed value may not be optimal, So I made the min distance relative to others, and updating it in each iteration for more precision:

```
//Mark all existing tracks as not used in this frame
vector<bool> used(tracks.size(), false);
double minDist = 20;
int matchedIdx = -1;
for (size_t i = 0; i < tracks.size(); i++) {
    if (used[i]) continue; // already matched
    double dist = norm(tracks[i].centroid - centroid);
    if (dist < minDist && dist < 50) { // threshold distance
        minDist = dist;
        matchedIdx = (int)i;
    }
}
```

If a match is found, the track is updated; otherwise, a new track is created and in both cases it's marked as used:

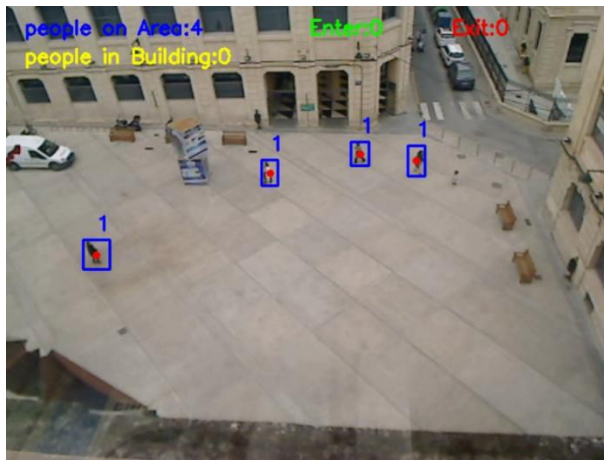
```
if (matchedIdx != -1) {
    Track& t = tracks[matchedIdx];
    used[matchedIdx] = true;
    //Update properties...
} else {
    Track newTrack;
    used.push_back(true);
    //Initialize track properties...
    tracks.push_back(newTrack);
    used.push_back(true);
}
```

7. Track Lifecycle Management

To handle occlusions and temporary detection losses, tracks are not immediately deleted. Instead, tracks are marked as missing if not updated and removed only after a defined threshold 3 frames:

```
for (size_t i = 0; i < tracks.size(); ++i) {
    if (!used[i])
        tracks[i].missingFrames++;
    else
        tracks[i].missingFrames = 0;
}

tracks.erase(remove_if(tracks.begin(), tracks.end(),
    [](const Track& t) {
        return t.missingFrames > 3;
    }), tracks.end());
```



This tracking pipeline allows the system to continuously monitor people in the scene and forms the basis for both **counting individuals in the square** and **determining entry/exit events** at the building entrances.

People Counting in the Square

To estimate the number of people present in the square at any given moment, the system uses a static binary mask (areaMask) corresponding to the visible square region in the video frame. Each tracked object is evaluated based on whether its centroid lies within this region.

The centroid of a tracked person is used as a representative point for presence. If the pixel at that location in the areaMask is above a threshold (i.e., within the white area of the mask), the person is considered to be inside the square.

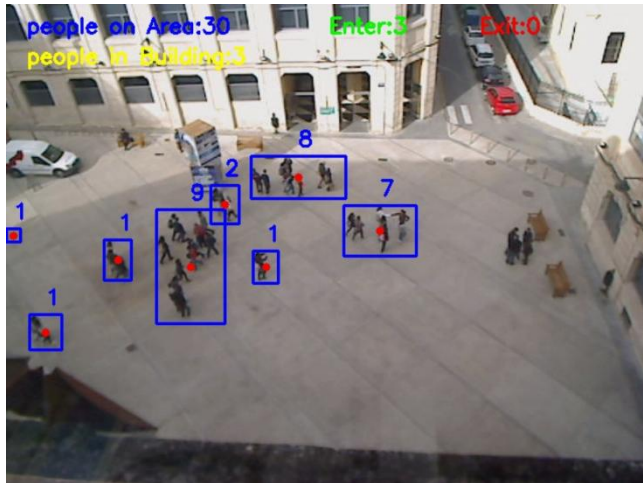
```
pixel = areaMask.at<uchar>(t.centroid.y, t.centroid.x);

if (pixel > 128) //object in plaza
    //increase count by the estimated number of people in the track
    n += t.estimatedPeople;
```

This approach ensures that only people physically located within the square are counted, excluding those outside the scene boundaries. The use of a binary mask simplifies the spatial classification and makes the system resilient to background clutter or ambiguous object locations.

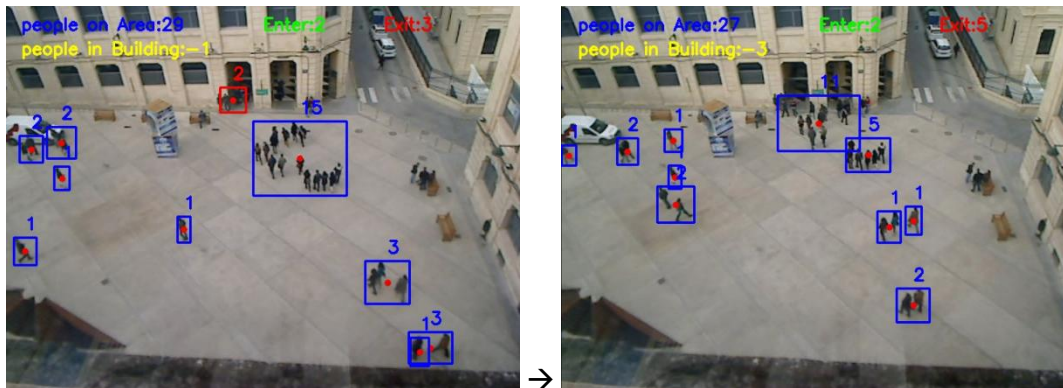
Test Cases

Blue Rectangle detects that the object is on the square. (Green detects entering door and red exiting it, we will talk about it later). In the top left corner of the video there's clearly shown the people count on the Plaza.



this test case shows that the tracking mechanism tracks all the moving object and separates them by contours. Later, the program depending on the contour area counts estimated number of people inside it and displays it on the video. Which are then summed up and displayed for users.

Down test cases clearly shows that the person detection, as well as counting people inside works well and is crucial in the cases when people move in groups. Logic is easy, but highly improves the accuracy.



One is to count people and another is not to count other moving objects in the square. As it's explained above this is done by size thresholding, assuming that there won't be a lot of people moving next to each other and by the idea that human shape isn't rectangle, therefore the bounding rectangle mustn't be filled. Test case down shows that the truck on video is excluded from the tracking:



Obviously only images are not enough for test cases, so here's the link to the video that shows how well the tracking algorithm detects and counts people on the plaza:

<https://drive.google.com/file/d/1bpYIIQ2pmY9mngIJrAICRdDVASZV8ZS/view?usp=sharing>

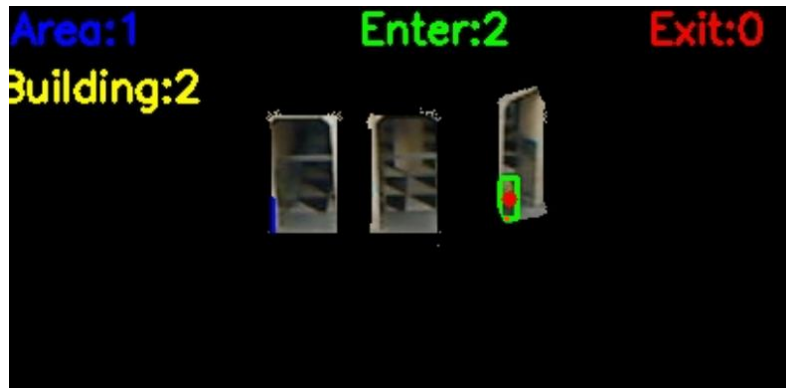
Entry and Exit Detection

A separate static mask (doorsMask) defines the door regions through which people enter and exit the Carbonell building. Entry/exit events are detected by observing the movement of tracked individuals across the mask boundary, using the bottom center point (middleDown) of their bounding boxes. Each track stores both the current and previous middleDown positions, allowing the system to detect transitions into or out of the door region. Basically, if middleDown point of person goes from outside the door to inside, so he enters the building, otherwise he exits.

```
uchar pixel = doorsMask.at<uchar>(t.middleDown.y, t.middleDown.x);
uchar pixelPrev = doorsMask.at<uchar>(t.prevMiddleDown.y,
t.prevMiddleDown.x);

//Only count if not already counted
if (!t.counted) {
    if (pixel < 128 && pixelPrev > 128) {
        people_exit += t.estimatedPeople; // (exited)
        t.counted = true;
    } else if (pixelPrev < 128 && pixel > 128) {
        people_enter += t.estimatedPeople; // (entered)
        t.counted = true;
    }
}
```

This directional logic allows the system to determine the flow of people through the doors, which, when combined with the initial building occupancy (assumed or initialized), enables continuous estimation of the number of people inside the Carbonell building.



This image shows that the person on right enters the building and it is detected because the middle down point (shown in orange) is going inside door mask.

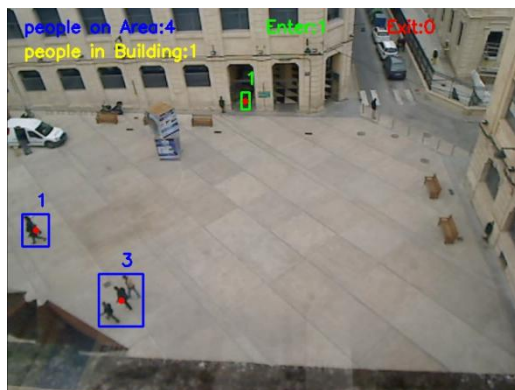
Test Cases

Green Rectangle detects entering the building and **red** exiting it. The video clearly displays count of both: entering and exiting. There's also shown estimated number inside the building which is simple difference between the number of people who entered and number of people who left.

With this algorithm, one of the first concerns that might come up is what happens if people are moving too near to the door, or directly in front of it. The precision of fixed mask and taking middle down point of the object (not centroid) ensures that the entrance/exit is only detected while it really happens. And the Video below proves this thesis:

https://drive.google.com/file/d/12ljPpYEJLZd2Nb0vXxPCqnRTw_YorZSe/view?usp=sharing

Detection of Enter:



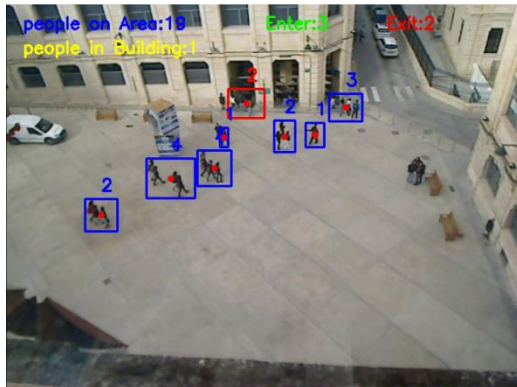
Video Example:

https://drive.google.com/file/d/1xU9LhLtGMN_UW_1IyIldMwuc4MIP-DW0/view?usp=sharing

below Video is especially interesting because it shows how a person separates from the group and enters the building, which is correctly detecting. Working with people moving together was one of the hardest parts, which I believe was successfully solved:

<https://drive.google.com/file/d/1-AJBhTib5nS5vwfatniVmRXEQpsXgNh-/view?usp=sharing>

Detection of exit: (in this case 2 people exit the door and thanks to our counting system it detect the number correctly)



Video Example:

https://drive.google.com/file/d/1s2_SMIzfRoOrGbpp1aiQtJ0Kl_6ikHAa/view?usp=sharing

This test cases have shown that detecting entering and exiting using tracked objects and doors Mask is straightforward task, which if implemented correctly gives us highly accurate results.

Final Word

To provide a comprehensive view of the system's performance across all test scenarios, a compiled video has been created using the footage from all provided assignment test cases. The video has been intentionally sped up to facilitate faster review while still clearly demonstrating the detection, tracking, and counting processes in action.

You can view the compiled result here:

<https://drive.google.com/file/d/1sXmttKWZolS0XozuXjb4ltOgvDqco1y/view?usp=sharing>

This final demonstration confirms that the core objectives of the project:

- counting people in the square
- detecting entries and exits through the Carbonell building doors
- estimating occupancy

have been successfully met.